

Elastic Resource Provisioning in Cloud Platforms via Workload Prediction and Performance Modeling

by

Qixu Chen

THESIS

Presented to

The Chinese University of Hong Kong, Shenzhen

in Fulfillment of the Requirements

for the Course of

CSC6032

School of Data Science / Computer Science Program

The Chinese University of Hong Kong, Shenzhen

April 2026

Abstract

Cloud computing relies on elasticity to adapt resource capacity to dynamic workload variations while maintaining quality of service (QoS) and service level agreement (SLA) requirements. However, practical auto-scaling remains challenging because virtual machine startup and reconfiguration operations introduce non-negligible delays, making purely reactive strategies insufficient for bursty and time-varying demand. To address this issue, this thesis presents an integrated view of prediction-driven cloud auto-scaling by combining workload forecasting, scaling trigger design, and resource provisioning. Specifically, representative approaches based on time-series analysis, neural networks, linear regression, Kalman filtering, pattern matching, support vector machine regression, and queueing-theoretic provisioning are synthesized. The resulting perspective highlights a complete pipeline in which monitored system metrics are first transformed into workload forecasts, then used to trigger scaling decisions ahead of demand changes, and finally mapped to resource allocation policies that balance response time and cost. By unifying these complementary ideas, this thesis provides a structured framework for understanding how predictive models can improve elasticity mechanisms in cloud environments.

Keywords: cloud computing, elasticity, auto-scaling, workload prediction, performance modeling, resource provisioning

Contents

Abstract	i
1 Introduction	1
2 Background	4
2.1 Resource Provisioning and Auto-scaling	4
2.2 Time-series-based Workload Prediction	5
2.3 Machine-learning-based Workload Prediction	6
3 Classical Workload Prediction Approaches for Cloud Elasticity	7
3.1 Classical Workload Prediction Methods	7
3.1.1 Time-Series Models	7
3.1.2 Kalman-Filter-Based Prediction	8
3.1.3 Pattern-Matching-Based Prediction	9
3.2 Pattern-Matching-Based Trigger Mechanism	10
3.3 Evaluation and Results Analysis	11
3.3.1 Experimental Setup and Evaluation Metric	11
3.3.2 Prediction Results of Different Models	12
3.3.3 Comparative Analysis and Discussion	14
3.4 Chapter Summary	15
4 Foundational Neural Workload Prediction Approaches for Cloud Elasticity	17
4.1 Foundational Neural and Regression-Based Workload Prediction	17
4.2 Sliding-Window-Based Prediction Strategy	19
4.3 Evaluation and Result Analysis	20
4.3.1 Experimental Setup	20
4.3.2 Prediction Results	22

4.3.3	Discussion	23
4.4	Chapter Summary	24
5	SLA-Aware Machine-Learning Provisioning Approaches for Cloud Elasticity	26
5.1	Support Vector Regression for Time-Series Workload Forecasting . . .	26
5.2	Queueing-Theoretic Performance Model	27
5.3	Evaluation and Result Analysis	29
5.3.1	Experimental Setup	29
5.3.2	Forecasting and Provisioning Results	32
5.3.3	Discussion	36
5.4	Chapter Summary	37
6	Comparative Analysis and Open Challenges	38
6.1	Methodological Progression Across the Reviewed Studies	38
6.2	Empirical Implications for Auto-Scaling	39
6.3	Open Challenges for Deployment-Oriented Elasticity	40
6.4	Summary	41
7	Conclusion	42

List of Figures

1.1	General workflow of a predictive elasticity mechanism for cloud auto-scaling. The system first collects monitoring data from virtual machines and hosts, then performs workload forecasting, and finally uses a trigger strategy to activate the elasticity mechanism.	3
3.1	Preprocessing of the pattern matching model. The observed CPU workload series is first transformed into a difference series and then converted into a string representation.	9
3.2	Evaluation results of time series approach	13
3.3	Evaluation results of Kalman Filter approach	13
3.4	Evaluation results of Pattern Matching approach	14
5.1	M/M/ c queue model for the distributed server. This figure illustrates a system with c parallel servers ($c > 1$), each serving incoming user requests. All requests join a single queue and are processed in order of arrival. λ denotes the arrival rate, i.e., the average number of requests arriving per unit time (modeled as a Poisson process), and μ denotes the service rate, i.e., the average number of requests that each server can process per unit time.	28

List of Tables

3.1	Error of Prediction	15
4.1	Data collection scenario.	20
4.2	Summary of evaluation metrics.	22
4.3	Comparison of prediction accuracy between NN and LR models with and without sliding window.	22
5.1	Selected values for the SVR parameters.	31
5.2	Summary of forecasting models used in the evaluation.	31
5.3	M/M/c queueing model parameters.	32
5.4	Forecasting accuracy of different models over the 24-hour test interval. .	33
5.5	SLA violations and unserved requests for the first resource allocation. .	34
5.6	Over-provisioned and under-provisioned resources in the first allocation.	35
5.7	SLA violations and unserved requests for the second resource allocation.	35
6.1	Methodological progression of predictive elasticity approaches.	39
6.2	Representative empirical findings from the reviewed studies.	40

Chapter 1

Introduction

Cloud computing has become a widely used platform because it offers on-demand services and a pay-as-you-go pricing model [30,47]. Instead of buying fixed infrastructure in advance, users can obtain computing resources when needed and release them when demand decreases. This flexibility makes cloud systems more cost-effective and easier to manage. A key capability behind this model is *elasticity*, which means that the system can automatically adjust its resources at runtime to match workload changes as closely as possible [20]. Good elasticity helps service providers maintain Quality of Service (QoS), satisfy Service Level Agreements (SLAs), and avoid unnecessary resource waste.

In practice, cloud elasticity can be achieved in several ways, including horizontal scaling, vertical scaling, and virtual machine migration. Among these, horizontal scaling is the most common in public cloud platforms, where virtual machines or service instances are added or removed according to demand. However, even this popular solution has an important limitation: starting a new virtual machine often takes several minutes [24]. Because of this delay, purely reactive auto-scaling methods may respond too late when workload rises suddenly. This can lead to overloaded systems, SLA violations, poor response time, or even service unavailability [19,34]. On the other hand, allocating too many resources for safety leads to over-provisioning, higher cost, and unnecessary energy consumption [5,15].

To address this problem, many researchers have studied predictive or proactive auto-scaling. Instead of waiting until the system is already under pressure, predictive methods estimate future workload from historical observations and provision resources in advance [32,34]. Accurate workload prediction is therefore very important for dy-

dynamic resource provisioning, capacity planning, and workload management in cloud environments. Existing studies have explored several forecasting approaches, including linear time-series models, regression methods, neural networks, and machine learning techniques [8, 27, 32]. Linear statistical models are simple and efficient, but they may not capture nonlinear workload patterns well. Neural-network-based methods can model nonlinear behavior, but they may require careful architecture design and expensive training [27]. Other studies have shown that machine learning methods such as support vector regression can provide strong forecasting accuracy while keeping the model relatively stable and practical for cloud auto-scaling [44].

In general, the goal of this line of research is to reduce scaling delay, improve QoS, and satisfy user SLAs. As shown in Fig. 1.1, the overall workflow mainly includes three steps. First, the system collects monitoring data from the cloud environment, such as CPU usage, memory usage, and disk-related metrics. Second, it analyzes the historical monitoring data and predicts future workload trends. Third, according to the predicted results, the trigger strategy decides when and how to activate the elasticity mechanism, so that resources can be provisioned or released in advance.

Although previous work has made important progress, there is still a need for prediction methods that are accurate, simple, and suitable for real-time elasticity control. Motivated by this need, this paper studies workload prediction for cloud elasticity and focuses on improving automatic scaling decisions through proactive forecasting. By predicting future workload ahead of resource setup time, the cloud system can reduce scaling delay, improve QoS, and make better use of resources.

The main contributions of this paper are summarized as follows:

- This paper summarizes the main ideas of workload prediction for cloud elasticity and proactive auto-scaling in cloud computing environments.
- It reviews several representative approaches introduced in previous studies, including statistical prediction methods, Kalman-filter-based prediction, pattern matching methods, regression models, neural-network-based forecasting, and support vector machine (SVM)-based prediction.

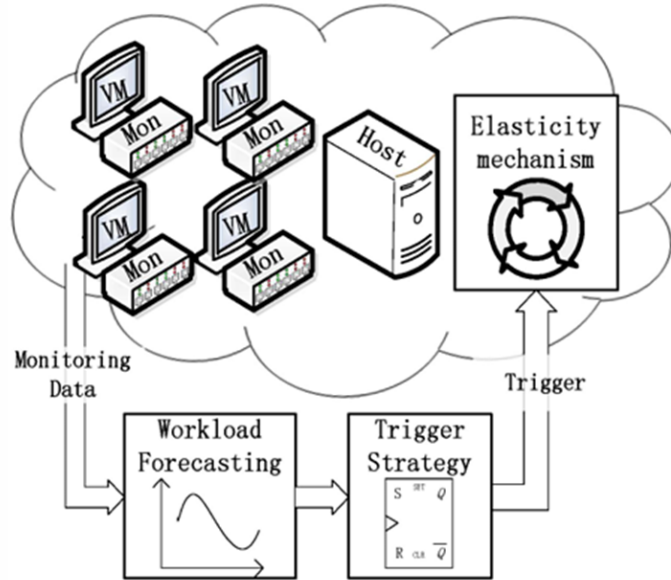


Figure 1.1: General workflow of a predictive elasticity mechanism for cloud auto-scaling. The system first collects monitoring data from virtual machines and hosts, then performs workload forecasting, and finally uses a trigger strategy to activate the elasticity mechanism.

- It further summarizes how prediction models can be combined with practical scaling strategies and performance models to support better resource allocation decisions in elastic cloud services.

Chapter 2

Background

2.1 Resource Provisioning and Auto-scaling

Resource provisioning is one of the core problems in cloud computing. Modern cloud applications are expected to provide stable Quality of Service (QoS) while operating under highly variable workloads. From the provider's perspective, over-provisioning leads to unnecessary operational cost, because excessive virtual machines or containers remain idle for a large portion of time. From the user's perspective, under-provisioning may cause long response time, service degradation, or even violations of Service Level Agreements (SLAs). Therefore, an effective resource provisioning mechanism should dynamically adjust computing resources according to application demand, while maintaining a balance between performance and cost.

Early studies on cloud and grid resource management mainly focused on autonomic and dynamic allocation of virtual resources. Van *et al.* [52] studied autonomic virtual resource management for service hosting platforms. Their work considered both SLA constraints and operating cost, and showed that resource management should not only react to performance degradation but also consider the economic impact of resource allocation decisions. Quiroz *et al.* [37] proposed an autonomic provisioning mechanism for enterprise grids and clouds. By incorporating workload modeling, their approach attempted to improve allocation decisions under changing application demand. Silva *et al.* [45] developed a heuristic for machine allocation in utility computing infrastructures, where the main goal was to improve resource utilization while satisfying application requirements. Lim *et al.* [31] discussed automated control in cloud computing and summarized important challenges and opportunities for feedback-based resource

management in cloud data centers.

These studies showed that dynamic provisioning is more effective than static resource allocation. However, many of these methods are mainly reactive, which means that they trigger scaling only after workload changes are observed. As a result, they are still affected by VM startup delay and may not respond quickly enough to sudden traffic changes.

2.2 Time-series-based Workload Prediction

To overcome the limitations of reactive scaling, many researchers have explored time-series methods for workload prediction. The basic idea of time-series-based prediction is to use historical monitoring data to estimate future workload or resource demand. In cloud platforms, such data may include CPU utilization, memory consumption, request arrival rate, network throughput, disk I/O, or application-level metrics such as response time and transaction rate. These methods use historical monitoring data to identify workload trends and forecast future demand.

Khan *et al.* [28] characterized cloud workload patterns using multiple time series and Hidden Markov Models. Hoffmann *et al.* [21] discussed empirical prediction methods such as autoregressive models for computing systems. Jiang *et al.* [26] proposed ASAP, a self-adaptive prediction system for instant cloud resource demand provisioning. Roy *et al.* [42] developed a predictive auto-scaling approach based on ARMA models. Other studies also applied moving average, autoregression, ARIMA, and related methods for cloud workload forecasting [3, 41, 55].

Time-series methods are usually simple, efficient, and easy to interpret. However, their performance may be limited when workload patterns are highly nonlinear or when the system behavior changes quickly over time. However, time-series methods also have several limitations. First, many classical models assume that workload patterns are stable or can be described by linear temporal relationships, which may not hold for highly nonlinear cloud applications. Second, they are often sensitive to sudden work-

load spikes. Third, their parameters usually need careful selection, and a model that works well for one application may not generalize to another. Therefore, although time-series methods are effective for regular and predictable workloads, their performance may decrease under highly dynamic conditions.

2.3 Machine-learning-based Workload Prediction

Machine learning is another important direction for cloud workload prediction. Compared with classical time-series methods, machine-learning-based methods often have stronger ability to model nonlinear relationships in monitoring data. Instead of relying only on explicit temporal assumptions, machine learning models can learn prediction patterns from training data. This makes them suitable for cloud applications whose workload behavior is affected by multiple interacting factors.

Islam *et al.* [24] developed empirical prediction models for adaptive resource provisioning using neural networks and linear regression. Imam *et al.* [22] applied neural network and regression methods to processor load prediction for efficient scaling of grid and cloud resources. Bankole and Ajila [4] compared support vector regression, neural networks, and linear regression for cloud client prediction.

In addition, some studies proposed pattern-based or hybrid forecasting methods. Caron *et al.* [10] used pattern matching to identify similar historical workload traces and predict future demand. Later studies also explored autonomic prediction systems and proactive scaling methods based on machine learning or hybrid models [18, 25, 36, 56]. These methods generally provide better flexibility for complex workloads, although they may require more training effort and parameter tuning.

Chapter 3

Classical Workload Prediction Approaches for Cloud Elasticity

3.1 Classical Workload Prediction Methods

3.1.1 Time-Series Models

Time-series models predict future workload from historical monitoring data, such as CPU utilization or request rate. This approach treats the monitored workload as a time series and applies mathematical models to estimate future values. Common models include moving average (MA), autoregression (AR), autoregressive integrated moving average (ARIMA), difference model (DM), and median model (MM).

The MA model predicts the future workload using past random error terms, while the AR model estimates the future value from previous workload observations. In general, an AR model can be written as

$$x_t = \phi_0 + \phi_1 x_{t-1} + \phi_2 x_{t-2} + \cdots + \phi_p x_{t-p} + \epsilon_t,$$

where x_t denotes the workload at time t , ϕ_i are model parameters, and ϵ_t is a white-noise error term. ARIMA further combines autoregression, moving average, and differencing, so it is more suitable for non-stationary workload series than simple AR or MA models.

The difference model (DM) is based on the AR model but first transforms the original workload series into a difference series:

$$y_t = x_t - x_{t-1}.$$

The AR prediction is then applied to the difference series. Since differencing can make the workload sequence closer to a stationary series, DM may improve prediction accuracy in some cases. The median model (MM) is a filter-based method. It uses the median value within a sliding window as the filtered value, which helps reduce the influence of abnormal or noisy data points.

Overall, time-series models are simple, efficient, and easy to implement. They can capture regular workload trends from historical data. However, because some models rely on smoothing, differencing, or filtering, they may lose part of the original workload variation. Their prediction accuracy may also decrease when the workload changes suddenly or becomes highly irregular.

3.1.2 Kalman-Filter-Based Prediction

The Kalman filter estimates the true workload from observed data. It includes two main steps: prediction and update. In the prediction step, the current state is estimated from the previous state and process noise. In the update step, the model uses the current observation to compute the Kalman gain and revise the state estimate.

Algorithm 1 Kalman Filter Model

Require: Observed CPU workload series Z , process covariance Q , observation covariance R , estimation covariance P

Ensure: Predicted workload series X

- 1: Initialize P , Q , and R
 - 2: **while** input is not null **do**
 - 3: $X = A \cdot X$
 - 4: $P = A \cdot P \cdot A^T + Q$
 - 5: $K_g = \frac{P \cdot H^T}{H \cdot P \cdot H^T + R}$
 - 6: $X = X + K_g \cdot (Z - H \cdot X)$
 - 7: $P = (I - K_g \cdot H) \cdot P$
 - 8: **end while**
 - 9: **return** X
-

This method supports real-time prediction. It can also reduce the effect of abnormal data and update the prediction adaptively through the Kalman gain. Therefore, it may achieve better accuracy than basic time-series models.

3.1.3 Pattern-Matching-Based Prediction

The pattern matching model predicts future workload by comparing the current sequence with historical patterns. In this method, string matching and Euclidean distance are used to find the most similar pattern. The whole process includes two steps: preprocessing and matching.

In the preprocessing step, the observed CPU workload series is first converted into a difference series, and then transformed into a string representation. This makes it easier to compare the current workload pattern with historical patterns. The preprocessing process is shown in Fig. 3.1.

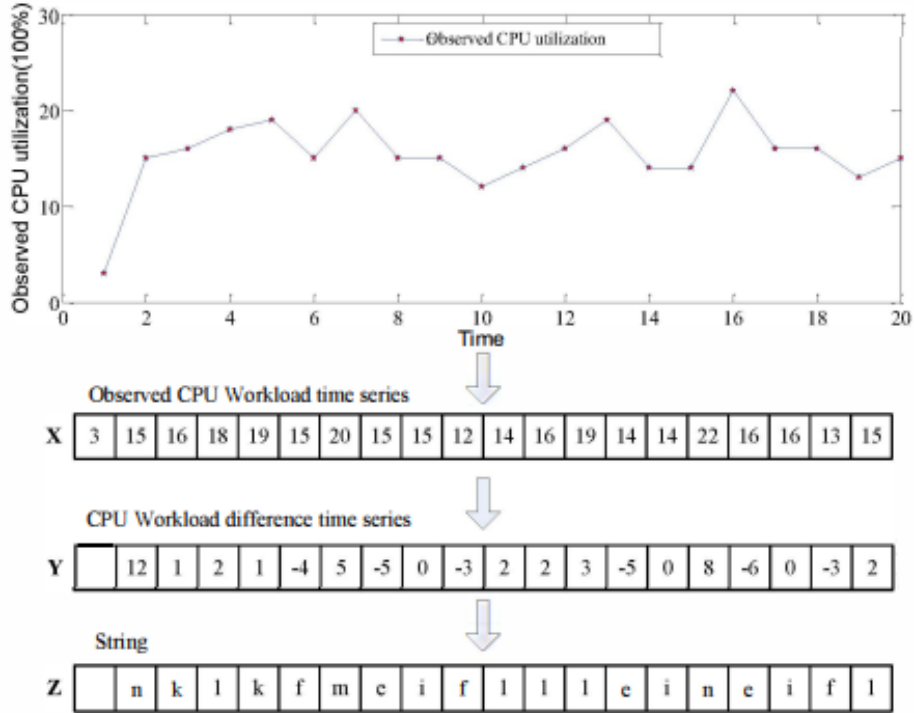


Figure 3.1: Preprocessing of the pattern matching model. The observed CPU workload series is first transformed into a difference series and then converted into a string representation.

In the matching step, the model compares the given string with the historical pattern set C . If an exact match exists, the corresponding pattern is returned directly. Otherwise, the model computes the similarity between the current string and historical strings, and selects the most similar one. In this paper, KMP string matching and Euclidean distance are used in the matching process.

Algorithm 2 Pattern Matching Algorithm

Require: Given string s , historical pattern set C

Ensure: Predicted pattern $C[num]$

```
1: Initialize distance array  $dis$ 
2: while  $num \neq 0$  do
3:    $length = C[num]$ 
4:    $tag = \text{KMP}(s, C[num])$ 
5:   if  $tag \neq -1$  then
6:     return  $C[num]$ 
7:   else
8:      $dis[num] = \infty$ 
9:     for  $i = 1$  to  $length - 1$  do
10:       $dis_1 = \text{Euclidean distance between } s \text{ and } C[num][i]$ 
11:      if  $dis_1 < dis[num]$  then
12:         $dis[num] = dis_1$ 
13:      end if
14:    end for
15:  end if
16: end while
17:  $num = \text{index of the minimum value in } dis$ 
18: return  $C[num]$ 
```

This method can capture recurring workload patterns in historical data. However, its efficiency may decrease when the historical pattern set becomes large.

3.2 Pattern-Matching-Based Trigger Mechanism

This subsection presents a trigger strategy for elasticity control based on workload prediction. The proposed strategy leverages the prediction results obtained from the pattern-matching model described in Section 3.1.3.

The trigger mechanism takes as input the predicted pattern sequence $C[num]$ and the current CPU workload value X_I . Based on these inputs, it determines whether to scale up, scale down, or maintain the current state. The decision is made by jointly considering the predicted workload trend and the current utilization level.

The overall procedure is summarized in Algorithm 3.

The proposed strategy determines whether to trigger elasticity actions by combining the predicted workload trend (captured by the pattern sequence) and the current CPU

Algorithm 3 A New Trigger Strategy

Require: Predicted string $C[num]$, CPU workload time series value X_I , scaling-up threshold $threshold_{up}$, scaling-down threshold $threshold_{down}$

Ensure: Trigger tag

```
1: Run the matching algorithm and obtain  $C[num]$ 
2:  $len = \text{length}(C[num])$ 
3:  $index = \text{get the current index of } C[num]$ 
4: for  $t = index$  to  $len$  do
5:   if all  $C[num][t]$  are greater than 'i' and  $X_I > threshold_{up}$  then
6:     return 1
7:   else if all  $C[num][t]$  are less than 'h' and  $X_I < threshold_{down}$  then
8:     return -1
9:   else
10:    return 0
11:   end if
12: end for
```

utilization.

3.3 Evaluation and Results Analysis

3.3.1 Experimental Setup and Evaluation Metric

This section evaluates the performance of the proposed workload prediction approaches. The experiments were conducted on virtual machines provided by the Aliyun cloud platform. In the experimental environment, the elastic scaling service was used, and the selected virtual machine instance was `ecs.t1.small`, which provides 1 GB RAM, 1 Mbps bandwidth, and a 20 GB hard disk.

The workload datasets were collected in two stages. In the first stage, a standard word count program was executed to obtain the CPU workload time series. In the second stage, workloads from computational tasks, web applications, and memory-intensive applications were collected. These workloads were then transformed into historical pattern strings through the preprocessing method described previously. The generated pattern strings include scaling-up strings, scaling-down strings, and stable strings.

To evaluate the prediction accuracy of different models, the Mean Absolute Percent-

age Error (MAPE) was used as the evaluation metric. It is defined as follows:

$$MAPE = \frac{1}{n} \sum_{i=1}^n \left| \frac{X_i - F_i}{X_i} \right|, \quad (3.1)$$

where X_i represents the original workload value, F_i represents the predicted value, and n denotes the total number of observations. A lower MAPE value indicates better prediction accuracy.

3.3.2 Prediction Results of Different Models

This subsection presents the prediction results of the time series approach, Kalman Filter approach, and Pattern Matching approach. The experimental results are shown in Figures 3.2, 3.3, and 3.4, respectively.

Time Series Approach

Figure 3.2 shows the evaluation results of the time series prediction approaches, including ARIMA, MA, AR, DM, and MM models. Different models demonstrate different prediction characteristics.

The MA model smooths the original workload data and reduces short-term fluctuations. The AR model mainly depends on previous workload values, so its prediction results are strongly affected by recent historical data. The DM model produces relatively stable results because of the use of differencing. Compared with these individual models, the ARIMA model combines the advantages of AR, MA, and differencing models, which allows it to achieve better prediction performance. The MM model also performs well in filtering workload fluctuation and jitter.

However, the prediction results of time series approaches generally lag behind the original workload series. This delay becomes more obvious when the workload changes sharply, especially near sudden workload peaks.

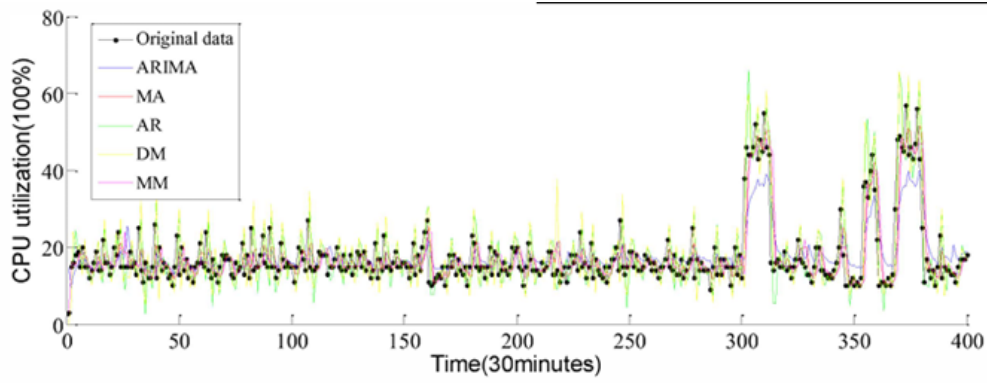


Figure 3.2: Evaluation results of time series approach

Kalman Filter Approach

Figure 3.3 presents the evaluation results of the Kalman Filter approach. The results show that the Kalman Filter model can effectively filter workload fluctuation and jitter. Its main contribution is to smooth the workload series and generate more stable prediction results.

However, the smoothing effect also causes the loss of some detailed workload changes. In addition, the Kalman Filter model requires an initial adaptation period to adjust its parameters. Similar to the time series approaches, the predicted workload series generated by the Kalman Filter model also shows a delay compared with the original workload series.

According to the experimental results, the Kalman Filter approach achieves a MAPE value of 0.2367, which is lower than the time series approaches. This indicates that the Kalman Filter approach provides better prediction accuracy in this experiment.

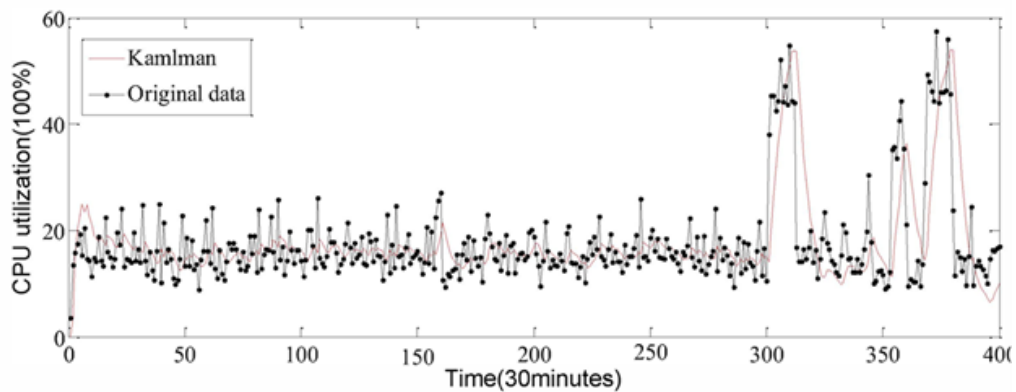


Figure 3.3: Evaluation results of Kalman Filter approach

Pattern Matching Approach

Figure 3.4 shows the evaluation results of the Pattern Matching approach. Compared with the time series and Kalman Filter approaches, the Pattern Matching approach performs better in capturing workload changing trends. It can follow the general direction of workload variation and is useful for identifying scaling-up, scaling-down, and stable workload patterns.

However, the prediction results are not very accurate near inflection points, where the workload changes suddenly. This is because the prediction performance of the Pattern Matching approach depends heavily on the completeness and quality of historical pattern strings. If the historical pattern string set is not sufficiently complete or representative, the model may fail to predict sudden workload changes accurately.

The MAPE value of the Pattern Matching approach is 0.2501. Although this value is slightly higher than that of the Kalman Filter approach, it is still lower than most traditional time series approaches. Therefore, the Pattern Matching approach shows strong potential in workload trend prediction.

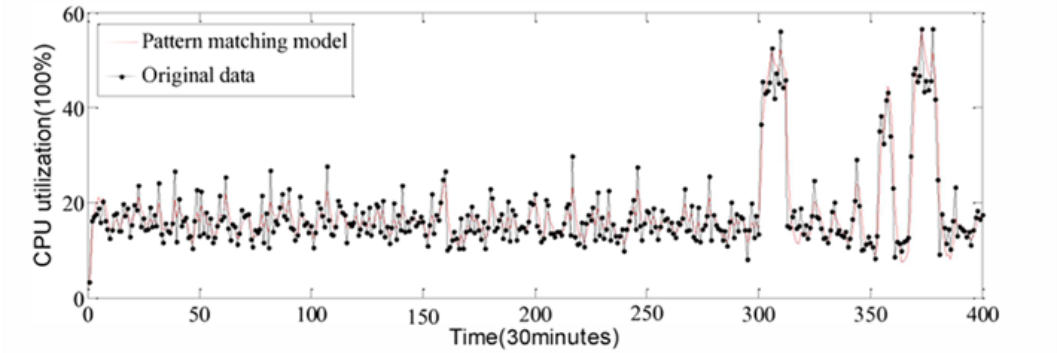


Figure 3.4: Evaluation results of Pattern Matching approach

3.3.3 Comparative Analysis and Discussion

Table 3.1 summarizes the prediction errors of different models using MAPE. The results show that the MM model and ARIMA model achieve better performance than the other time series approaches. Among them, the ARIMA model obtains a MAPE value of

0.2690, while the MM model obtains a MAPE value of 0.2744.

Table 3.1: Error of Prediction

	MA	AR	ARIMA	DM	MM
MAPE	0.2820	0.2958	0.2690	0.4504	0.2744
	Kalman Filter		Pattern Matching Model		
MAPE	0.2367		0.2501		

As shown in Table 3.1, the Kalman Filter approach achieves the lowest prediction error, with a MAPE value of 0.2367. The Pattern Matching approach obtains a MAPE value of 0.2501, which is slightly higher than that of the Kalman Filter approach but lower than those of the time series approaches. This indicates that both the Kalman Filter approach and the Pattern Matching approach provide better prediction accuracy than traditional time series models in this experiment.

The reason why the Pattern Matching approach performs slightly worse than the Kalman Filter approach may be that the historical pattern string set is not sufficiently complete or representative. Since the Pattern Matching approach relies heavily on historical patterns, insufficient pattern strings may reduce its prediction accuracy, especially near workload inflection points.

Overall, the experimental results demonstrate that the proposed models have good prediction performance. The Kalman Filter approach provides the best overall prediction accuracy, while the Pattern Matching approach shows strong capability in capturing workload changing trends. The time series approaches, especially ARIMA and MM, also achieve acceptable results, but they are less responsive to sudden workload changes.

3.4 Chapter Summary

This chapter studied classical workload prediction approaches for cloud elasticity. It first introduced several time-series models, including MA, AR, ARIMA, DM, and MM. It then described the Kalman filter model, which adaptively updates workload predictions through state estimation, and the pattern matching model, which predicts future

workload by searching similar historical workload patterns.

A pattern-matching-based elasticity trigger strategy was also presented. This strategy combines the predicted workload pattern with the current CPU utilization to determine whether the system should scale up, scale down, or maintain the current resource allocation. Compared with purely threshold-based mechanisms, this strategy considers both current workload status and predicted future trends.

The evaluation was conducted on virtual machines from the Aliyun cloud platform. The results show that the Kalman filter achieves the lowest MAPE value of 0.2367, while the pattern matching model achieves a MAPE value of 0.2501 and performs well in capturing workload trends. Among traditional time-series models, ARIMA and MM achieve relatively better results. These findings show that classical prediction methods can provide a useful foundation for proactive elasticity control, although their accuracy and responsiveness may still be limited under sudden and irregular workload changes.

Chapter 4

Foundational Neural Workload Prediction Approaches for Cloud Elasticity

4.1 Foundational Neural and Regression-Based Workload Prediction

Early workload prediction studies commonly formulate the problem as time-series forecasting, where future resource demand is estimated from historical observations. Representative foundational approaches include Error Correction Neural Networks (ECNN), Linear Regression, and the sliding window technique for transforming sequential traces into supervised learning samples [14, 49, 54].

ECNN is an early feed-forward neural model trained on historical workload data [49]. A typical three-layer neural network consists of an input layer, one or more hidden layers, and an output layer. For an input vector

$$\mathbf{x} = [x_1, x_2, \dots, x_m],$$

the network produces a predicted output by propagating the input values through weighted connections. During training, the synaptic weights are first initialized, the network generates a predicted value, and the prediction error is then used to adjust the weights. This iterative correction process gives the model its name.

For a training sample $\mathbf{x}^{(i)}$, the predicted output of neuron p can be expressed as

$$\hat{y}_p(i) = f \left(\sum_{j=1}^l w_{pj} x_j(i) \right),$$

where $f(\cdot)$ is the activation function, w_{pj} is the weight between input j and neuron p ,

and $x_j(i)$ is the j -th input value of sample i .

The prediction error is computed as

$$e_p(i) = \hat{y}_p(i) - y_p(i),$$

where $\hat{y}_p(i)$ is the predicted output and $y_p(i)$ is the actual output. The local gradient of neuron p is then given by

$$\delta_p(i) = e_p(i) f' \left(\sum_{j=1}^l w_{pj} x_j(i) \right).$$

Based on this gradient, the weight vector is updated as

$$\mathbf{w}_p(t+1) = \mathbf{w}_p(t) - \rho \sum_{i=1}^N \delta_p(i) \mathbf{x}(i),$$

where $\rho \in [0, 1]$ is the learning rate and N is the number of training samples. The training process continues until a termination condition is satisfied, for example when the total prediction error falls below a predefined threshold. This procedure corresponds to the back-propagation algorithm based on gradient descent [49].

Linear Regression provides a simpler and more interpretable alternative for workload prediction. It models the relationship between input variables and a dependent output variable through a linear equation [54]. Given an input vector

$$\mathbf{x} = [x_1, x_2, \dots, x_m],$$

the predicted output is defined as

$$y = \beta_0 + \sum_{i=1}^m \beta_i x_i,$$

where β_0 is the intercept and β_i denotes the regression coefficient associated with input variable x_i . In cloud workload prediction, Linear Regression is often used as a baseline model because it is simple, efficient, and easy to interpret. However, its linear

formulation limits its ability to capture nonlinear temporal patterns in resource usage.

4.2 Sliding-Window-Based Prediction Strategy

To apply neural and regression-based models to workload traces, the original time series must first be transformed into supervised learning samples. This transformation is commonly achieved through the sliding window technique [14].

Let $y(t)$ denote the resource usage value observed at time t . Given a window size k , the input vector is constructed from k consecutive observations:

$$\mathbf{x} = [y(t), y(t+1), \dots, y(t+k-1)].$$

The prediction target is a future value located r intervals ahead of the input window:

$$y(t+k+r-1).$$

Therefore, the original time-series forecasting problem is converted into a supervised learning problem, where recent historical observations are used as input features and a future workload value is used as the output label.

This strategy is particularly useful for cloud elasticity because auto-scaling decisions must be made before resource demand actually increases. Since virtual machine instances require several minutes to become available after being requested, the prediction horizon should be long enough to allow the system to provision additional resources in advance. In this chapter, the prediction interval is set to 12 minutes, which reflects the practical delay involved in cloud instance provisioning.

Two prediction settings are considered. In the first setting, the models predict CPU usage 12 minutes ahead using only the current resource usage value. This setting is used to train an ECNN model and a Linear Regression model. In the second setting, the sliding window technique is applied, so that multiple recent CPU usage values are used as input. Under this setting, ECNN is trained with multiple input neurons, while Linear

Regression is extended to Multiple Linear Regression.

4.3 Evaluation and Result Analysis

This section evaluates the foundational neural and regression-based workload prediction approaches described above. The evaluation focuses on predicting future cloud resource demand, especially CPU utilization, under a dynamically changing workload. The prediction target is the aggregated percentage of CPU usage across all running virtual machine instances.

4.3.1 Experimental Setup

The experimental data were collected from an e-commerce benchmark application deployed on Amazon EC2. The application was based on the TPC-W benchmark, which emulates the behavior of an online bookstore, including browsing, ordering, and transactional activities. To simulate a workload pattern similar to the ramp-up phase of flash-crowd traffic, the client workload was configured to generate requests in a linear fashion.

During the experiment, Amazon’s auto-scaling mechanism was used to provision EC2 instances according to predefined scaling rules. CPU utilization data were sampled every minute and aggregated across all running virtual machine instances. Since the metric represents the sum of CPU utilization across multiple instances, the aggregated CPU usage can exceed 100% when more than one instance is active.

Table 4.1: Data collection scenario.

Parameter	Value
Number of samples	135
Benchmark duration	135 min
Sampling interval	1 min
Sampled resource	Aggregated CPU usage across all VM instances
Maximum number of EC2 instances	3
Minimum number of EC2 instances	1

As shown in Table 4.1, the final data set contains 135 observations collected over

135 minutes, with one sample recorded every minute. The number of active EC2 instances varied between one and three during the experiment. The collected historical CPU usage data were then used to train and evaluate the prediction models.

Leave-One-Out Cross Validation (LOOCV) was used to evaluate the models. In LOOCV, each observation is used once as the testing sample, while the remaining $n - 1$ observations are used for training. This process is repeated for all observations, and the predicted values are compared with the actual values to measure model accuracy. LOOCV is suitable for this experiment because the data set contains a limited number of samples, allowing each data point to contribute to both training and validation.

The ECNN model was implemented using the `nnnet` package in R. The network used one hidden layer with seven hidden neurons and a learning rate of

$$\rho = 0.7.$$

The Linear Regression model was implemented using the `dynlm` package in R. For the sliding window setting, different window sizes from 2 to 9 were tested to examine how the number of historical observations affects prediction accuracy.

Four metrics are used to evaluate prediction accuracy: Mean Absolute Percentage Error (MAPE), PRED(25), Root Mean Squared Error (RMSE), and R^2 .

MAPE measures the average relative difference between predicted and actual values:

$$MAPE = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|.$$

RMSE measures the absolute prediction deviation and gives larger penalties to large errors:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}.$$

PRED(25) represents the proportion of predictions whose relative error is within 25%

of the actual value:

$$PRED(25) = \frac{1}{n} \sum_{i=1}^n I \left(\left| \frac{y_i - \hat{y}_i}{y_i} \right| \leq 0.25 \right),$$

where $I(\cdot)$ is an indicator function. Finally, R^2 measures how well the prediction model explains the variation in the observed CPU usage data:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}.$$

For MAPE and RMSE, lower values indicate better prediction accuracy. For PRED(25) and R^2 , higher values indicate better prediction performance.

Table 4.2: Summary of evaluation metrics.

Metric	Description	Preferred value
MAPE	Average relative prediction error	Lower
PRED(25)	Percentage of predictions within 25% relative error	Higher
RMSE	Absolute prediction deviation	Lower
R^2	Goodness of fit between predicted and actual values	Higher

4.3.2 Prediction Results

The prediction accuracy of the Neural Network (NN) and Linear Regression (LR) models was evaluated both with and without the sliding window strategy. Table 4.3 summarizes the results using MAPE, PRED(25), RMSE, and R^2 .

Table 4.3: Comparison of prediction accuracy between NN and LR models with and without sliding window.

Metric	Without sliding window		With sliding window					
	NN	LR	NN			LR		
			Size = 3	Size = 6	Size = 9	Size = 3	Size = 6	Size = 9
MAPE	0.195	0.364	0.197	0.186	0.193	0.365	0.377	0.381
PRED(25)	0.819	0.810	0.800	0.857	0.857	0.838	0.848	0.848
RMSE	9.60	10.41	9.81	10.15	9.67	10.24	10.05	9.94
R^2	–	0.954	–	–	–	0.955	0.955	0.954

Without the sliding window strategy, the NN model achieves better prediction accuracy than the LR model. The NN obtains a lower MAPE value of 0.195 compared

with 0.364 for LR, indicating a smaller relative prediction error. It also achieves a lower RMSE value, 9.60 compared with 10.41, and a slightly higher PRED(25) value, 0.819 compared with 0.810. These results indicate that the NN model captures the CPU usage trend more effectively when only the current resource usage value is used as input.

When the sliding window strategy is applied, the NN model still shows better overall performance than LR. For all reported window sizes, NN produces substantially lower MAPE values than LR. For example, when the window size is 6, the MAPE of NN is 0.186, while the MAPE of LR is 0.377. This suggests that NN benefits more from recent historical observations and is more effective at learning nonlinear patterns in cloud resource usage.

The sliding window size also affects prediction accuracy. For the NN model, increasing the window size generally improves PRED(25), which increases from 0.800 at window size 3 to 0.857 at window sizes 6 and 9. However, the improvement is not strictly linear. The lowest MAPE among the reported NN sliding-window settings is obtained when the window size is 6. For LR, the sliding window strategy slightly improves PRED(25), but MAPE increases as the window size becomes larger. This indicates that LR is less effective at exploiting longer historical input windows.

The R^2 metric is reported only for LR models, because it is mainly used to measure the goodness of fit in linear regression. Therefore, the comparison between NN and LR mainly relies on MAPE, PRED(25), and RMSE. Overall, the results show that NN provides more accurate and stable CPU usage prediction than LR, especially when an appropriate sliding window size is selected.

4.3.3 Discussion

The evaluation demonstrates that neural workload prediction can provide more accurate results than a simple linear model in a dynamic cloud environment. The NN model achieves lower relative and absolute prediction errors in most cases, showing its ability to capture nonlinear relationships between historical CPU usage and future demand.

The sliding window strategy plays an important role in improving prediction perfor-

mance. By using multiple recent observations as input, the prediction model can exploit short-term temporal patterns in the workload trace. However, the results also show that increasing the window size does not always guarantee better accuracy. For NN, a moderate window size of 6 provides the lowest MAPE, while window sizes 6 and 9 achieve the highest PRED(25). This suggests that the model benefits from additional historical context, but too much input history may introduce noise or redundant information.

For LR, the sliding window strategy does not lead to the same level of improvement. Although PRED(25) slightly increases with larger window sizes, MAPE becomes worse. This result is consistent with the limitations of linear models: when the relationship between past and future resource usage is nonlinear, adding more input variables does not necessarily improve prediction accuracy.

From an elasticity perspective, these results indicate that NN-based prediction can support proactive auto-scaling by forecasting CPU usage before additional resources are needed. Since the prediction interval is set to 12 minutes, the model provides enough time for the cloud platform to request and initialize new virtual machine instances before the workload increase becomes critical. Therefore, foundational neural prediction methods provide an important basis for later machine-learning-based cloud elasticity approaches.

4.4 Chapter Summary

This chapter studied foundational neural and regression-based workload prediction approaches for cloud elasticity. It first introduced ECNN and Linear Regression as representative early prediction models, and then explained how the sliding window technique transforms cloud workload traces into supervised learning samples.

The evaluation used CPU utilization data collected from an e-commerce benchmark application deployed on Amazon EC2. The experiment predicted aggregated CPU usage 12 minutes ahead using NN and LR models, both with and without the sliding window strategy. The results show that NN generally outperforms LR in terms of MAPE,

PRED(25), and RMSE. In particular, the NN model with a sliding window size of 6 achieves the lowest MAPE, while window sizes 6 and 9 achieve the best PRED(25).

Overall, this chapter shows that neural models are more effective than simple linear models for predicting dynamic cloud resource usage. The results also highlight the importance of selecting an appropriate historical input window. These findings provide a foundation for more advanced machine-learning-based workload forecasting and proactive resource provisioning methods discussed in later chapters.

Chapter 5

SLA-Aware Machine-Learning Provisioning Approaches for Cloud Elasticity

5.1 Support Vector Regression for Time-Series Workload Forecasting

As workload prediction gradually evolved from linear statistical modeling to nonlinear learning-based forecasting, artificial neural networks (ANNs) and support vector machines (SVMs) became two representative directions for time-series prediction [1, 2, 35, 50, 57]. Compared with conventional linear models, these methods are better suited to capturing the complex and nonlinear dynamics commonly observed in cloud workloads.

ANN-based forecasting methods are data-driven and self-adaptive, and thus do not require strong prior assumptions about the underlying data distribution [2, 57]. However, their training process may suffer from local minima and unstable optimization. In contrast, SVM-based methods are grounded in the structural risk minimization principle and generally provide a unique global solution, which makes them attractive for workload forecasting tasks [46, 53].

For a workload time series

$$s = \{s_1, s_2, \dots, s_N\}, \quad s_i \in \mathbb{R},$$

the sequence is first transformed into supervised samples through lagged windows. Given a window size T , each training input is constructed from previous observations, while the corresponding target is the next value in the series. Based on these input-

output pairs, support vector regression (SVR) predicts the future workload as

$$\hat{y}_t = b + \sum_{m=1}^M w_m K(\mathbf{x}_t, \mathbf{x}_m),$$

where b is the bias term, w_m are model coefficients, and $K(\cdot, \cdot)$ is the kernel function [35, 46, 50]. Through kernel mapping, SVR projects the original input space into a higher-dimensional feature space, where nonlinear forecasting can be addressed by linear regression [43].

The SVR model is typically learned by minimizing a regularized objective that balances model complexity and empirical prediction error. A widely used formulation is based on the ε -insensitive loss, which ignores errors within a tolerance band and penalizes only larger deviations [46, 53]. This mechanism improves generalization ability and makes SVR particularly suitable for noisy time-series environments. In practice, the forecasting performance of SVR depends strongly on the choice of hyperparameters, such as the penalty factor and kernel parameters [13].

Commonly used kernels include polynomial kernels and radial basis function (RBF) kernels, with the latter being especially popular in time-series forecasting due to its flexibility in modeling nonlinear patterns [43, 46]. Overall, SVR provides a compact yet theoretically grounded framework for workload prediction, and has become one of the most influential machine learning approaches for nonlinear time-series forecasting.

5.2 Queueing-Theoretic Performance Model

Based on the workload predictions obtained from the Support Vector Regression (SVR) model described in Section 5.1, the next step is to determine the amount of computing resources required to satisfy the performance constraints of the system. To this end, a queueing-theoretic performance model is adopted to map predicted workload into resource provisioning decisions.

We consider a distributed server architecture with a single entry point, where in-

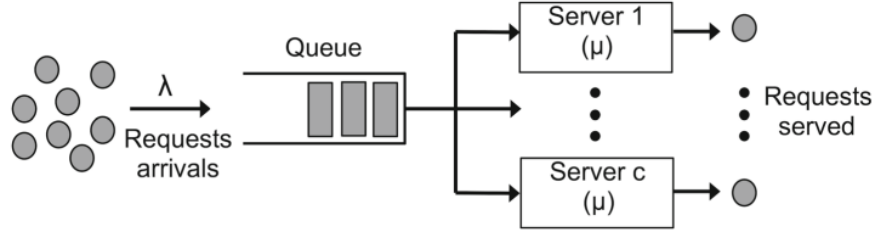


Figure 5.1: M/M/ c queue model for the distributed server. This figure illustrates a system with c parallel servers ($c > 1$), each serving incoming user requests. All requests join a single queue and are processed in order of arrival. λ denotes the arrival rate, i.e., the average number of requests arriving per unit time (modeled as a Poisson process), and μ denotes the service rate, i.e., the average number of requests that each server can process per unit time.

coming user requests are first queued and then dispatched by a load balancer to multiple parallel backend servers. Such a system can be modeled as an M/M/ c queue, as illustrated in Fig. 5.1.

In this model, c denotes the number of parallel servers, λ is the arrival rate of requests (modeled as a Poisson process), and μ is the service rate of each server. Based on these parameters, the system utilization is defined as

$$\rho = \frac{\lambda}{c\mu}.$$

To ensure system stability, the condition $\rho < 1$ must be satisfied.

The average waiting time in the queue is given by

$$W_q = \frac{c^c \rho^{c+1} P_0}{\lambda c! (1 - \rho)^2},$$

where P_0 is the probability that the system is idle:

$$P_0 = \left(\frac{c^c \rho^c}{c! (1 - \rho)} + \sum_{n=0}^{c-1} \frac{(c\rho)^n}{n!} \right)^{-1}.$$

Accordingly, the average response time is

$$W = W_q + \frac{1}{\mu}.$$

Given the predicted workload (i.e., estimated arrival rate λ from the SVR model), the objective is to determine the minimum number of servers c such that both stability and SLA requirements are satisfied:

$$\rho < 1, \quad W \leq W_{\text{SLA}}.$$

In this work, an auto-scaling period of one hour is considered. Specifically, the SVR model predicts the average workload for the next hour, and the M/M/ c model is then used to determine the required number of servers for that period. This setting avoids excessive resource fluctuations and aligns with the billing granularity of typical cloud providers. Nevertheless, the proposed framework can be extended to other scaling intervals depending on the requirements of the system.

5.3 Evaluation and Result Analysis

This section evaluates the proposed SLA-aware provisioning approach. The evaluation first analyzes the workload forecasting accuracy of different prediction models, and then studies how these predictions affect resource provisioning decisions under the M/M/ c performance model. The objective is to verify whether machine-learning-based forecasting can produce resource allocations closer to the optimal allocation obtained from the real workload.

5.3.1 Experimental Setup

The workload trace used in this evaluation was obtained from real web service logs collected at the Complutense University of Madrid. The data were gathered over a four-week period on an hourly basis. In order to emulate a cloud service with high traffic intensity, the original workload values were extrapolated by one order of magnitude. The resulting time series represents the hourly average server load, expressed in requests per second.

The training data set contains four weeks of observations, namely $N=672$ hourly samples. The forecasting model uses historical workload observations to predict the average load for the next hourly period. Since the workload trace shows a clear daily pattern, the autocorrelation of the input data was analyzed to determine the lag period. The autocorrelation result indicates a strong correlation at a lag interval of 24 hours. Therefore, the lag period is set to $T=24$. This means that each prediction is generated from the workload values observed during the previous 24 hours.

The forecasting horizon is set to one hour. This choice is appropriate for proactive auto-scaling because prediction accuracy generally decreases as the forecasting horizon increases. Moreover, an hourly horizon is consistent with the provisioning period considered in this work. For evaluation, hourly predictions are generated for a 24-hour test interval. Since the real workload values during this interval are known, the predicted values can be compared with the actual values.

The SVR model requires three important parameters: the regularization parameter C , the ε parameter of the ε -insensitive loss function, and the RBF kernel parameter γ . Following the analytical parameter selection method proposed by Cherkassky and Ma, the regularization parameter is selected according to the range of the target values in the training data:

$$C = \max(|\bar{y} + 3\sigma_y|, |\bar{y} - 3\sigma_y|),$$

where $\bar{y}$ and σ_y are the mean and standard deviation of the training target values, respectively.

The value of ε is selected according to the estimated noise level of the input data:

$$\varepsilon = 3\sigma\sqrt{\frac{\ln n}{n}},$$

where n is the number of training samples and σ is the estimated noise standard deviation. The noise variance is estimated by fitting the training data using a first-order linear regression model:

$$\sigma^2 = \frac{1}{n-1} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

Finally, the RBF kernel parameter is selected according to the range of the input data:

$$\gamma \sim [0.1, 0.5] \times \text{range}(x),$$

where

$$\text{range}(x) = \max(x) - \min(x).$$

Using the workload trace and the above parameter selection method, the selected SVR parameters are summarized in Table 5.1.

Table 5.1: Selected values for the SVR parameters.

Parameter	Value
C	1.9
ε	0.027
γ	[0.2, 1.0]

The SVR models were implemented using the WEKA tool with its time-series analysis package. To evaluate the benefit of SVR-based forecasting, the proposed models are compared with three simple baseline methods: last-value prediction, moving average prediction, and linear regression. In addition, several SVR kernels are tested, including polynomial, normalized polynomial, and RBF kernels. The complete set of forecasting models is shown in Table 5.2.

Table 5.2: Summary of forecasting models used in the evaluation.

Model	Method	Parameters
#1	Last value	—
#2	Moving average	MA(3)
#3	Linear regression	AR(24)
#4	SVM with polynomial kernel	$C = 1.9, \varepsilon = 0.027, p = 1$
#5	SVM with normalized polynomial kernel	$C = 1.9, \varepsilon = 0.027, p = 2$
#6	SVM with RBF kernel	$C = 1.9, \varepsilon = 0.027, \gamma = 0.2$
#7	SVM with RBF kernel	$C = 1.9, \varepsilon = 0.027, \gamma = 0.6$
#8	SVM with RBF kernel	$C = 1.9, \varepsilon = 0.027, \gamma = 1.0$

For the resource provisioning evaluation, the predicted workload is used as the arrival rate λ in the M/M/c queueing model. The service rate of each backend server is

assumed to be

$$\mu = 200 \text{ requests/s.}$$

The SLA constraint is defined in terms of the average response time. Since the service time of one backend server is $1/\mu = 5 \text{ ms}$, the maximum queueing time is set to

$$W_q \leq 2.5 \text{ ms.}$$

Therefore, the maximum average response time allowed by the SLA is

$$W = W_q + \frac{1}{\mu} \leq 7.5 \text{ ms.}$$

The queueing model parameters are summarized in Table 5.3.

Table 5.3: M/M/c queueing model parameters.

Parameter	Meaning	Value
λ	Arrival rate	Real or predicted server load
μ	Service rate	200 requests/s
ρ	System utilization	< 1
W	Average response time	$\leq 7.5 \text{ ms}$
W_q	Average queueing time	$\leq 2.5 \text{ ms}$

For each hourly period, the provisioning algorithm searches for the minimum number of backend servers c that satisfies both the stability condition and the SLA constraint:

$$\rho = \frac{\lambda}{c\mu} < 1,$$

$$W \leq 7.5 \text{ ms.}$$

5.3.2 Forecasting and Provisioning Results

The forecasting accuracy is evaluated over the 24-hour test interval. Three standard error metrics are considered: Mean Absolute Error (MAE), Mean Squared Error (MSE),

and Root Mean Squared Error (RMSE):

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|,$$

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2,$$

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}.$$

Here, y_i denotes the real workload value, $\hat{y}_i$ denotes the predicted workload value, and n is the number of observations in the test interval.

The forecasting errors are summarized in Table 5.4. The results show that the SVR-based models generally achieve lower forecasting errors than the three baseline methods. In particular, the RBF-kernel SVR models provide strong prediction accuracy, while the normalized polynomial kernel also performs well.

Table 5.4: Forecasting accuracy of different models over the 24-hour test interval.

Forecasting model	MAE	RMSE
Fcast. #1	0.135	0.167
Fcast. #2	0.134	0.164
Fcast. #3	0.115	0.137
Fcast. #4	0.113	0.138
Fcast. #5	0.098	0.116
Fcast. #6	0.094	0.115
Fcast. #7	0.098	0.120
Fcast. #8	0.094	0.113

After obtaining the workload predictions, the M/M/c queueing model is used to estimate the number of backend servers required for each hour. The estimated allocation is compared with the optimal allocation computed from the real workload. If the estimated allocation is larger than the optimal allocation, the system is over-provisioned. If it is smaller than the optimal allocation, the system is under-provisioned. If both values are equal, the system is correctly provisioned.

Three evaluation metrics are used to assess the quality of resource allocation: the number of provisioned resources, the percentage of SLA violations, and the percentage

of unserved requests. A request is considered to violate the SLA when its response time is greater than 7.5 ms. In addition, a timeout of 1 s is assumed; if the response time exceeds this limit, the request is considered unserved.

The first resource allocation experiment uses the central predicted workload values as input to the M/M/c model. Table 5.5 shows the percentage of SLA violations and unserved requests for each forecasting model. Although the SVR-based models produce allocations closer to the optimal number of resources in many cases, all forecasting models still lead to a relatively high number of SLA violations. This is mainly caused by under-provisioning during several hourly periods.

Table 5.5: SLA violations and unserved requests for the first resource allocation.

Resource allocation	SLA violations (%)	Unserved requests (%)
Optimal	0.4	0.4
Fcast. #1	46.0	5.4
Fcast. #2	41.9	5.4
Fcast. #3	41.1	3.6
Fcast. #4	44.3	4.5
Fcast. #5	44.5	3.1
Fcast. #6	47.9	3.7
Fcast. #7	45.3	3.8
Fcast. #8	44.1	3.2

The over-provisioning and under-provisioning results for the first allocation are summarized in Table 5.6. The SVR-based models require fewer over-provisioned resources than the baseline methods. Forecasting model #6, which uses an RBF kernel with $\gamma = 0.2$, produces the lowest number of over-provisioned resources. However, under-provisioning remains significant, which explains the high percentage of SLA violations.

To reduce the number of SLA violations, a second resource allocation experiment is performed. Instead of using only the central predicted workload value, the input load is increased by half of the expected prediction error:

$$\lambda_t^+ = \hat{\lambda}_t + \frac{1}{2}e_t,$$

where $\hat{\lambda}_t$ is the predicted load and e_t denotes the expected forecasting error. This strat-

Table 5.6: Over-provisioned and under-provisioned resources in the first allocation.

Forecasting model	Over-provisioned resources	Under-provisioned resources
Fcast. #1	79	82
Fcast. #2	79	80
Fcast. #3	77	61
Fcast. #4	63	72
Fcast. #5	60	56
Fcast. #6	50	68
Fcast. #7	56	65
Fcast. #8	56	57

egy intentionally adds a safety margin to reduce the risk of under-provisioning.

The results of the second allocation are shown in Table 5.7. Compared with the first allocation, the percentage of SLA violations is substantially reduced for all forecasting models. The best result is obtained by forecasting model #5, which uses the normalized polynomial kernel. This model achieves 8.0% SLA violations and only 0.1% unserved requests, showing the best overall trade-off among the evaluated models.

Table 5.7: SLA violations and unserved requests for the second resource allocation.

Resource allocation	SLA violations (%)	Unserved requests (%)
Optimal	0.4	0.4
Fcast. #1	11.2	0.4
Fcast. #2	15.2	0.7
Fcast. #3	10.8	0.1
Fcast. #4	17.6	0.5
Fcast. #5	8.0	0.1
Fcast. #6	19.3	0.5
Fcast. #7	34.6	1.5
Fcast. #8	33.6	1.3

These results indicate that prediction accuracy alone is not the only factor that determines provisioning quality. A model with very low forecasting error may still cause SLA violations if its errors frequently lead to under-provisioning. Therefore, an effective auto-scaling mechanism must balance three objectives: minimizing the number of allocated resources, minimizing SLA violations, and minimizing unserved requests.

5.3.3 Discussion

The evaluation confirms that SVR-based forecasting models provide better workload prediction accuracy than simple forecasting methods. The RBF-kernel SVR models achieve the lowest MAE and RMSE values, which demonstrates their ability to capture nonlinear workload patterns. The normalized polynomial kernel also performs competitively and provides the best overall provisioning result when the prediction margin is added.

From the resource provisioning perspective, the results show that forecasting errors have a direct impact on SLA compliance. In the first allocation experiment, even small under-estimations of the workload lead to under-provisioning, which significantly increases the percentage of SLA violations. By adding half of the expected prediction error to the forecasted workload, the second allocation strategy reduces the risk of under-provisioning and substantially improves SLA compliance.

However, this improvement comes at the cost of additional resource usage. Adding a safety margin increases the probability of over-provisioning, especially during periods when the workload is over-estimated. Therefore, the provisioning strategy must carefully balance performance and cost. For latency-sensitive services, a conservative allocation may be preferable because SLA violations can lead to penalties and user dissatisfaction. For cost-sensitive services, a smaller safety margin may be more appropriate.

Overall, the best trade-off is achieved by the SVR model with the normalized polynomial kernel. This model provides a good balance between resource usage, SLA violations, and unserved requests. The results also suggest that a proactive auto-scaling mechanism based on workload forecasting and queueing-theoretic resource estimation can allocate resources more effectively than simple prediction-based methods.

Another important practical observation is that the proposed auto-scaling mechanism has low computational overhead. Both workload forecasting and resource estimation can be completed within a short time, typically less than one minute. Since

cloud instances can also be provisioned or de-provisioned within a few minutes, the auto-scaler can be executed shortly before the next hourly period begins. This allows the required resources to become available in time for the next provisioning interval.

5.4 Chapter Summary

This chapter studied an SLA-aware machine-learning provisioning approach for elastic cloud services. The approach combines SVR-based workload forecasting with an M/M/ c queueing model. The SVR model predicts the future request rate from historical workload observations, while the queueing model converts this predicted workload into the minimum number of backend servers required to satisfy response-time constraints.

The evaluation shows that SVR-based forecasting models generally outperform simple forecasting methods such as last-value prediction, moving average, and linear regression. Among the SVR models, RBF kernels achieve strong prediction accuracy, while the normalized polynomial kernel provides the best trade-off in resource allocation when a prediction safety margin is considered. The results demonstrate that combining machine-learning forecasting with queueing-theoretic performance modeling is an effective strategy for proactive cloud auto-scaling.

Nevertheless, the approach also has limitations. The M/M/ c model assumes Poisson arrivals and exponentially distributed service times, which may not fully represent all real cloud applications. In addition, the forecasting model relies on historical patterns and may be less accurate under sudden workload changes. Future extensions may consider adaptive retraining, dynamic safety margins, and more general performance models for complex distributed services such as Hadoop and Spark clusters.

Chapter 6

Comparative Analysis and Open Challenges

6.1 Methodological Progression Across the Reviewed Studies

The reviewed approaches show a methodological progression from workload signal prediction to decision-oriented provisioning.

The first group consists of classical prediction methods. Time-series models, Kalman filtering, and pattern matching use historical CPU workload traces to estimate future workload behavior. These methods are relatively lightweight and suitable for online monitoring environments. Their main contribution is to provide earlier workload signals for elasticity control.

The second group introduces learning-based prediction. Neural Networks and Linear Regression are used to predict future CPU utilization from historical resource usage. The sliding window technique further transforms sequential observations into supervised learning samples. Compared with classical methods, this approach provides stronger learning capability, especially when workload behavior is nonlinear.

The third group connects workload forecasting with resource provisioning. In the SVR-based approach, the predicted request rate is used as the input of an $M/M/c$ queueing model. The model then estimates the number of backend servers required to satisfy response-time constraints. Therefore, this approach produces not only a workload forecast, but also a resource allocation decision.

Table 6.1: Methodological progression of predictive elasticity approaches.

Level	Main methods	Prediction target	Elasticity role
Classical prediction	ARIMA, Kalman filter, pattern matching	CPU workload value or workload trend	Supports earlier trigger decisions
Learning-based prediction	Neural Network, Linear Regression, sliding window	Future CPU utilization	Supports proactive VM provisioning
Decision-oriented provisioning	SVR and M/M/c queueing model	Request rate and required server number	Supports SLA-aware allocation

6.2 Empirical Implications for Auto-Scaling

The experimental results indicate that more expressive models usually improve prediction performance, but prediction accuracy alone does not fully determine the quality of an auto-scaling mechanism.

In the classical prediction study, the Kalman filter achieves the lowest MAPE value of 0.2367. The pattern matching model obtains a MAPE value of 0.2501 and performs well in capturing workload trends. These results suggest that adaptive filtering and pattern-based prediction can improve accuracy compared with basic time-series models.

In the neural prediction study, Neural Networks generally outperform Linear Regression. The best reported Neural Network setting uses a sliding window size of 6 and achieves a MAPE value of 0.186. This result shows that recent historical observations can improve short-term CPU usage prediction when they are used as structured input features.

In the SVR-based study, SVR models, especially those using RBF kernels, achieve lower forecasting errors than last-value, moving average, and linear regression baselines. More importantly, this study evaluates the operational effect of prediction errors. Under-estimation may lead to under-provisioning and SLA violations, while over-estimation may increase infrastructure cost. Thus, workload prediction should be assessed not only by statistical error, but also by its impact on resource allocation.

Table 6.2: Representative empirical findings from the reviewed studies.

Study type	Best or representative result	Main implication
Classical prediction	Kalman filter: MAPE = 0.2367; pattern matching: MAPE = 0.2501	Adaptive filtering and pattern recognition improve basic workload prediction.
Learning-based prediction	Neural Network with sliding window size 6: MAPE = 0.186	Nonlinear learning benefits from recent historical observations.
SVR-based provisioning	RBF-kernel SVR outperforms last-value, moving average, and linear regression baselines	Prediction errors must be evaluated according to their provisioning consequences.

6.3 Open Challenges for Deployment-Oriented Elasticity

Several challenges remain when applying predictive elasticity mechanisms to real cloud systems.

First, the reviewed studies use different platforms, workloads, and evaluation metrics. Hu et al. evaluate CPU workload traces on Aliyun, Islam et al. use the TPC-W benchmark on Amazon EC2, and Moreno-Vozmediano et al. use real web service request logs. These differences make direct numerical comparison difficult. More unified benchmarks are needed for fair evaluation.

Second, most approaches focus on a single workload indicator, such as CPU utilization or request rate. Real cloud applications may be limited by multiple resources, including memory, disk I/O, network bandwidth, and database performance. Future work should consider multi-resource prediction and multi-tier application behavior.

Third, workload patterns may change over time. Flash crowds, seasonal effects, software updates, and failures can make historical patterns unreliable. This requires adaptive retraining, online learning, or concept drift detection.

Fourth, many models generate point predictions without uncertainty estimation. For auto-scaling, uncertainty is important because under-estimation during peak periods can cause serious SLA violations. Risk-aware provisioning strategies are therefore needed.

Finally, prediction should be integrated with cost and SLA objectives. A practi-

cal auto-scaler should not only forecast demand accurately, but also determine resource allocation policies that balance SLA compliance, infrastructure cost, and system robustness.

6.4 Summary

This chapter compared three categories of predictive elasticity approaches. Classical methods provide lightweight workload prediction and trend detection. Neural approaches improve CPU usage forecasting by learning from historical observations, especially when sliding windows are used. SVR-based provisioning further connects workload forecasting with queueing theory, allowing predicted request rates to be translated into SLA-aware server allocation.

Overall, the reviewed studies show that cloud elasticity is moving from workload estimation toward decision-oriented provisioning. The central challenge is to convert uncertain workload predictions into scaling decisions that satisfy SLA requirements while avoiding unnecessary over-provisioning.

Chapter 7

Conclusion

Cloud elasticity is an important feature of modern cloud computing systems. It allows applications to add or remove resources according to workload changes. However, reactive auto-scaling is not always enough because new virtual machines or backend servers need time to start. If the system only scales after overload happens, users may experience slow response time, service degradation, or SLA violations. Therefore, proactive elasticity, which predicts future workload before making scaling decisions, is important for improving cloud resource management.

This report reviewed three studies on predictive elasticity. The first study uses time-series methods, Kalman filtering, and pattern matching to predict CPU workload and improve the trigger timing of auto-scaling. The second study uses Neural Networks and Linear Regression to predict future resource demand. It applies the sliding window technique to transform historical CPU usage into training data, so that the system can forecast workload before new virtual machines are needed. The third study uses Support Vector Regression to predict request rates and combines the prediction result with an M/M/c queueing model. In this way, it can estimate the number of backend servers required to satisfy response-time requirements.

The comparison shows a clear development from workload prediction to resource allocation. Classical prediction methods are simple and efficient, but they may not capture complex workload patterns very well. Learning-based methods are more powerful and can better handle nonlinear workload changes. Decision-oriented methods go further because they connect prediction results with actual provisioning decisions. This makes them more useful for SLA-aware auto-scaling.

The reviewed results also show that prediction accuracy is important, but it is not

the only factor. In auto-scaling, prediction errors directly affect system performance and cost. If the workload is underestimated, the system may allocate too few resources, which can cause SLA violations. If the workload is overestimated, the system may allocate too many resources, which increases cost. Therefore, predictive elasticity should be evaluated not only by statistical errors such as MAPE, RMSE, or MAE, but also by its impact on response time, resource usage, cost, and SLA satisfaction.

There are still several challenges for practical deployment. First, different studies use different platforms, workloads, and evaluation metrics, so direct comparison is difficult. Second, many approaches focus only on CPU utilization or request rate, while real cloud applications may also be affected by memory, disk I/O, network bandwidth, and database performance. Third, workload patterns may change over time because of flash crowds, seasonal changes, software updates, or failures. This means prediction models should be able to adapt to new workload patterns. Finally, many models only provide point predictions and do not consider uncertainty. In real systems, uncertainty is important because underestimation during peak periods can cause serious service problems.

In conclusion, proactive elasticity is a promising solution to the limitations of reactive auto-scaling. The reviewed studies show that workload prediction can help cloud systems make earlier and better scaling decisions. However, the final goal is not only to predict workload accurately, but also to support reliable, cost-effective, and SLA-aware resource allocation. Future work should combine multi-resource prediction, adaptive learning, uncertainty estimation, and cost-SLA optimization to make predictive elasticity more practical for real cloud environments.

References

- [1] N. Ahmed, A. Atiya, N. E. Gayar, and H. El-Shishiny, “An empirical comparison of machine learning models for time series forecasting,” *Econometric Reviews*, vol. 29, no. 5–6, pp. 594–621, 2010.
- [2] H. Allende, C. Moraga, and R. Salas, “Artificial neural networks in time series forecasting: a comparative analysis,” *Kybernetika*, vol. 38, no. 6, pp. 685–707, 2002.
- [3] F. J. Almeida Morais, F. V. Brasileiro, R. V. Lopes, P. H. Gomes, and J. M. Nogueira, “Autoflex: Service agnostic auto-scaling framework for IaaS deployment models,” in *Proc. IEEE/ACM Int. Symp. Cluster, Cloud and Grid Computing*, 2013, pp. 42–49.
- [4] A. A. Bankole and S. A. Ajila, “Cloud client prediction models for cloud resource provisioning in a multitier web application environment,” in *Proc. IEEE 7th Int. Symp. Service-Oriented System Engineering*, 2013, pp. 156–161.
- [5] A. Beloglazov, J. Abawajy, and R. Buyya, “Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing,” *Future Generation Computer Systems*, vol. 28, no. 5, pp. 755–768, 2012.
- [6] M. Boardman and T. Trappenberg, “A heuristic for free parameter optimization with support vector machines,” in *Proc. IEEE Int. Joint Conf. Neural Networks*, 2006, pp. 610–617.
- [7] G. Bontempi, S. B. Taieb, and Y. Le Borgne, *Business Intelligence: Second European Summer School, eBISS 2012, Brussels, Belgium, July 15–21, 2012*. Berlin, Heidelberg: Springer, 2013.

- [8] G. E. P. Box and G. M. Jenkins, *Time Series Analysis, Forecasting and Control*. San Francisco, CA, USA: Holden-Day, 1990.
- [9] C. Burges, “A tutorial on support vector machines for pattern recognition,” *Data Mining and Knowledge Discovery*, vol. 2, no. 2, pp. 121–167, 1998.
- [10] E. Caron, F. Desprez, and A. Muresan, “Forecasting for cloud computing on-demand resources based on pattern matching,” INRIA, Technical Report, 2010.
- [11] M. Cassabaum, D. Waagen, J. Rodriguez, and H. Schmitt, “Unsupervised optimization of support vector machine parameters,” in *Proc. SPIE 5426, Automatic Target Recognition XIV*, Bellingham, WA, USA: SPIE, 2004, pp. 316–325.
- [12] O. Chapelle, V. Vapnik, O. Bousquet, and S. Mukherjee, “Choosing multiple parameters for support vector machines,” *Machine Learning*, vol. 46, no. 1, pp. 131–159, 2002.
- [13] V. Cherkassky and Y. Ma, “Practical selection of SVM parameters and noise estimation for SVM regression,” *Neural Networks*, vol. 17, no. 1, pp. 113–126, 2004.
- [14] T. Dietterich, “Machine learning for sequential data: a review,” in *Structural, Syntactic, and Statistical Pattern Recognition*, T. Caelli, A. Amin, R. Duin, J. de Ridder, and M. Kamel, Eds. Berlin, Heidelberg: Springer, 2002, pp. 227–246.
- [15] B. Dougherty, J. White, and D. C. Schmidt, “Model-driven auto-scaling of green cloud computing infrastructure,” *Future Generation Computer Systems*, vol. 28, no. 2, pp. 371–378, 2012.
- [16] B. Efron and G. Gong, “A leisurely look at the bootstrap, the jackknife, and cross-validation,” *The American Statistician*, vol. 37, no. 1, pp. 36–48, 1983.
- [17] E. Frank, M. Hall, G. Holmes, R. Kirkby, and B. Pfahringer, *WEKA—A Machine Learning Workbench for Data Mining*. Boston, MA, USA: Springer, 2005.

- [18] Z. Gong, X. Gu, and J. Wilkes, "PRESS: Predictive elastic resource scaling for cloud systems," in *Proc. Int. Conf. Network and Service Management*, 2010, pp. 9–16.
- [19] A. M. Halavais, *The Slashdot Effect: Analysis of a Large-Scale Public Conversation on the World Wide Web*. Ph.D. dissertation, Univ. Washington, Seattle, WA, USA, 2001.
- [20] N. Herbst, S. Kounev, and R. Reussner, "Elasticity in cloud computing: What it is, and what it is not," in *Proc. 10th Int. Conf. Autonomic Computing (ICAC)*, San Jose, CA, USA, 2013, pp. 23–27.
- [21] G. Hoffmann, K. S. Trivedi, and M. Malek, "A best practice guide to resource prediction for computing systems," *IEEE Trans. Rel.*, vol. 56, no. 4, pp. 615–628, 2007.
- [22] M. T. Imam, S. F. Miskhat, R. M. Rahman, and M. Kaykobad, "Neural network and regression based processor load prediction for efficient scaling of grid and cloud resources," in *Proc. 14th Int. Conf. Computer and Information Technology*, 2011, pp. 333–338.
- [23] W. Iqbal, M. Dailey, D. Carrera, and P. Janecek, "Adaptive resource provisioning for read intensive multi-tier applications in the cloud," *Future Generation Computer Systems*, vol. 27, no. 6, pp. 871–879, 2011.
- [24] S. Islam, J. Keung, K. Lee, and A. Liu, "Empirical prediction models for adaptive resource provisioning in the cloud," *Future Generation Computer Systems*, vol. 28, no. 1, pp. 155–162, 2012.
- [25] J. J. Jheng, F. H. Tseng, H. C. Chao, and L. D. Chou, "A novel VM workload prediction using Grey prediction model in cloud data center," in *Proc. Int. Conf. Information Networking*, 2014, pp. 40–45.

- [26] Y. Jiang, C. Perng, T. Li, and R. Chang, “ASAP: A self-adaptive prediction system for instant cloud resource demand provisioning,” in *Proc. IEEE 11th Int. Conf. Data Mining*, 2011, pp. 1104–1109.
- [27] Y. Jiang, C. Perng, T. Li, and R. Chang, “Self-adaptive cloud capacity planning,” in *Proc. IEEE 9th Int. Conf. Services Computing*, 2012, pp. 73–80.
- [28] A. Khan, X. Yan, S. Tao, and N. Anerousis, “Workload characterization and prediction in the cloud: A multiple time series approach,” in *Proc. IEEE Network Operations and Management Symp.*, 2012, pp. 1287–1294.
- [29] L. Kujawa, “Performance benchmark of popular PHP frameworks,” 2013.
- [30] A. Li, X. Yang, S. Kandula, and M. Zhang, “CloudCmp: comparing public cloud providers,” in *Proc. Internet Measurement Conf.*, 2010, pp. 1–14.
- [31] H. C. Lim, S. Babu, J. S. Chase, and S. S. Parekh, “Automated control in cloud computing: Challenges and opportunities,” in *Proc. 1st Workshop on Automated Control for Datacenters and Clouds*, 2009, pp. 13–18.
- [32] T. Llorido-Botran, J. Miguel-Alonso, and J. A. Lozano, “A review of auto-scaling techniques for elastic applications in cloud environments,” *Journal of Grid Computing*, vol. 12, no. 4, pp. 559–592, 2014.
- [33] F. Manco, C. Lupu, F. Schmidt, J. Mendes, S. Kuenzer, S. Sati, K. Yasukata, C. Raiciu, and F. Huici, “My VM is lighter (and safer) than your container,” in *Proc. ACM Symp. Operating Systems Principles*, 2017, pp. 218–233.
- [34] R. Moreno-Vozmediano, R. S. Montero, and I. M. Llorente, “Efficient resource provisioning for elastic cloud services based on machine learning techniques,” *Journal of Cloud Computing*, vol. 8, no. 5, 2019.
- [35] K.-R. Müller, A. Smola, G. Rätsch, B. Schölkopf, J. Kohlmorgen, and V. Vapnik, “Predicting time series with support vector machines,” in *Proc. 7th Int. Conf. Artificial Neural Networks (ICANN)*, Lausanne, Switzerland, 1997, pp. 999–1004.

- [36] A. Y. Nikraves, S. A. Ajila, and C. H. Lung, “Towards an autonomic auto-scaling prediction system for cloud resource provisioning,” in *Proc. IEEE/ACM Int. Symp. Software Engineering for Adaptive and Self-Managing Systems*, 2015, pp. 35–45.
- [37] A. Quiroz, H. Kim, M. Parashar, N. Gnanasambandam, and N. Sharma, “Towards autonomic workload provisioning for enterprise grids and clouds,” in *Proc. 10th IEEE/ACM Int. Conf. Grid Computing*, 2009, pp. 50–57.
- [38] T. Rauber and K. Berns, *Kernel Multilayer Perceptron*. Washington, DC, USA: IEEE Computer Society, 2011.
- [39] K. Razavi, G. van der Kolk, and T. Kielmann, “Prebaked μ VMs: scalable, instant VM startup for IaaS clouds,” in *Proc. IEEE Int. Conf. Distributed Computing Systems*, 2015, pp. 245–255.
- [40] K. Razavi, S. Costache, A. Gardiman, K. Verstoep, and T. Kielmann, “Scaling VM deployment in an open source cloud stack,” in *Proc. Workshop on Scientific Cloud Computing*, 2015.
- [41] G. Reig and J. Guitart, “On the anticipation of resource demands to fulfill the QoS of SaaS web applications,” in *Proc. ACM/IEEE Int. Conf. Grid Computing*, 2012, pp. 147–154.
- [42] N. Roy, A. Dubey, and A. Gokhale, “Efficient autoscaling in the cloud using predictive models for workload forecasting,” in *Proc. IEEE Int. Conf. Cloud Computing*, 2011, pp. 500–507.
- [43] J. Shawe-Taylor and N. Cristianini, *Kernel Methods for Pattern Analysis*. New York, NY, USA: Cambridge University Press, 2004.
- [44] S. Shevade, S. Keerthi, C. Bhattacharyya, and K. Murthy, “Improvements to the SMO algorithm for SVM regression,” *IEEE Trans. Neural Netw.*, vol. 11, no. 5, pp. 1188–1193, 2000.

- [45] J. A. N. Silva, L. Veiga, and P. Ferreira, “Heuristic for resources allocation on utility computing infrastructures,” in *Proc. 6th Int. Workshop on Middleware for Grid Computing*, 2008, pp. 1–6.
- [46] A. Smola and B. Schölkopf, “A tutorial on support vector regression,” *Statistics and Computing*, vol. 14, pp. 199–222, 2004.
- [47] S. T. T. Soares, J. M. Gomes, and D. G. Sa, “CloudReports: uma ferramenta grafica para a simulacao de ambientes computacionais em nuvem baseada no framework CloudSim,” 2011.
- [48] J. Suykens and J. Vandewalle, “Least squares support vector machine classifiers,” *Neural Processing Letters*, vol. 9, no. 3, pp. 293–300, 1999.
- [49] S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed. Amsterdam, The Netherlands: Academic Press, 2008.
- [50] U. Thissen, R. van Brakel, A. de Weijer, W. Melssen, and L. Buydens, “Using support vector machines for time series prediction,” *Chemometrics and Intelligent Laboratory Systems*, vol. 69, no. 1–2, pp. 35–49, 2003.
- [51] B. Üstün, W. Melssen, and L. Buydens, “Facilitating the application of support vector regression by using a universal Pearson VII function based kernel,” *Chemometrics and Intelligent Laboratory Systems*, vol. 81, no. 1, pp. 29–40, 2006.
- [52] N. Van, H. D. Tran, and J.-M. Menaud, “Autonomic virtual resource management for service hosting platforms,” in *Proc. ICSE Workshop on Software Engineering Challenges of Cloud Computing*, 2009, pp. 1–8.
- [53] V. Vapnik, *Statistical Learning Theory*. New York, NY, USA: Wiley-Interscience, 1998.
- [54] S. Weisberg, *Applied Linear Regression*, 3rd ed. Hoboken, NJ, USA: Wiley, 2005.

- [55] W. Xu, X. Zhu, S. Singhal, and Z. Wang, “Predictive control for dynamic resource allocation in enterprise data centers,” in *Proc. IEEE/IFIP Network Operations and Management Symp.*, 2006, pp. 115–126.
- [56] J. Yang, C. Liu, Y. Shang, B. Cheng, Z. Mao, C. Chen, and J. Lloret, “Workload forecasting-based automatic scaling in service clouds,” in *Proc. IEEE 6th Int. Conf. Cloud Computing*, 2013, pp. 810–815.
- [57] G. Zhang, “Time series forecasting using a hybrid ARIMA and neural network model,” *Neurocomputing*, vol. 50, pp. 159–175, 2003.